PYTHON II 2

F-Strings και
η Δομή match

F-Strings

- Τα F-Strings στην Python, γνωστά και ως Formatted String Literals, είναι ένας τρόπος για να δημιουργείτε δυναμικά Αλφαριθμητικά, ενσωματώνοντας Μεταβλητές και Εκφράσεις μέσα σε αυτά, με συνοπτικό και ευανάγνωστο τρόπο αυξάνοντας την Αναγνωσιμότητα του κώδικα και μειώνοντας το Μέγεθος του σημαντικά.
- Προστέθηκαν στην Python 3.6 και είναι πλέον η προτιμώμενη μέθοδος για την μορφοποίηση κειμένου.

F-Strings

- Για να δημιουργήσετε ένα F-String, θα πρέπει να προηγείται του Αλφαριθμητικού το γράμμα 'f' ή 'F' πριν ακριβός από το άνοιγμα των Εισαγωγικών.
- Μέσα στο Αλφαριθμητικό, μπορείτε να τοποθετήσετε Εκφράσεις μέσα σε Αγκύλες {} που ονομάζονται Θέσεις Περιεχομένου.
- Αυτές οι Εκφράσεις εκτελούνται από την Python κατά την εκτέλεση του κώδικα και αντικαθιστά το περιεχόμενο των Αγκύλων με τα αποτελέσματά τους.

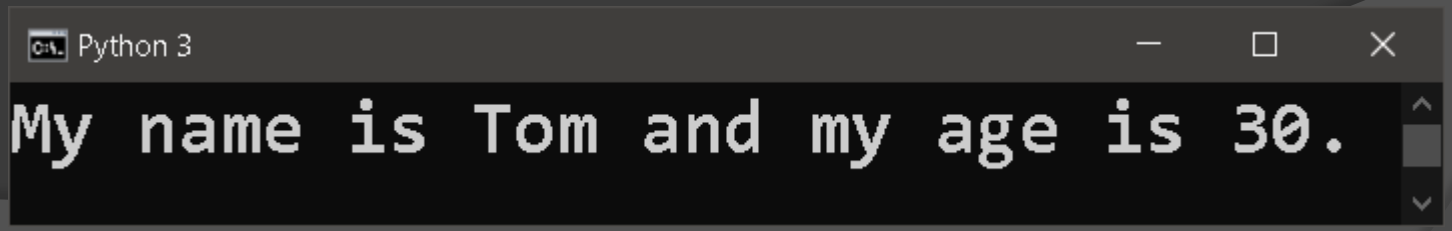
F-Strings

Συντακτικό:

```
print( f"κείμενο" )
```

Παράδειγμα:

```
name = "Tom"
age = 30
print( f"My name is {name} and
                                    my age is {age}." )
```



F-Strings

- Τα βασικότερα Πλεονεκτήματα των F-String είναι:

1. Ευαναγνωσιμότητα.
2. Συνοπτικότητα.
3. Ταχύτητα.
4. Ευελιξία.
5. Δυνατότητες Μορφοποίησης.

F-Strings

Ευαναγνωσιμότητα:

- Τα F-Strings είναι πιο εύκολα στην ανάγνωση και στην χρήση από άλλες μεθόδους μορφοποίησης Αλφαριθμητικών, όπως η str.format() ή η χρήση του τελεστή +.

Συνοπτικότητα:

- Επιτρέπουν την απευθείας ενσωμάτωση Εκφράσεων χωρίς την ανάγκη για πρόσθετες ενέργειες, όπως η Μετατροπή Τύπου ή η χρήση πολλών και διαφορετικών Τελεστών.

F-Strings

Ταχύτητα:

- Συνήθως είναι πιο γρήγορα ως αναφορά την εκτέλεση των Ενεργειών από άλλες μεθόδους μορφοποίησης Αλφαριθμητικών όπως είδαμε και προηγούμενος.

Ευελιξία:

- Μπορούν να ενσωματώσουν διάφορους Τύπους Δεδομένων μέσα στο Αλφαριθμητικό, όπως Αριθμούς, Λίστες, Λεξικά και Εκφράσεις χωρίς επιπλέον χειρισμούς.

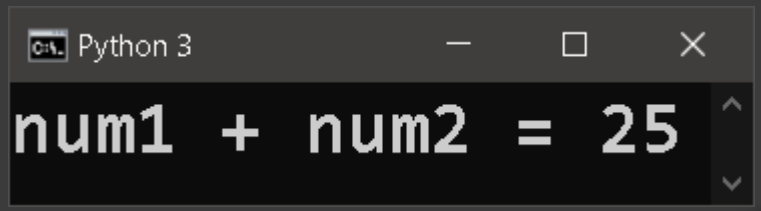
F-Strings

Δυνατότητες Μορφοποίησης:

- Υποστηρίζουν διάφορες Επιλογές Μορφοποίησης για να προσαρμόσετε την εμφάνιση των τιμών, όπως Ευθυγράμμιση, Περιθώρια, και Αριθμητική Μορφοποίηση.

Παράδειγμα:

```
num1 = 10
num2 = 15
print( f"num1 + num2 = {num1 + num2}." )
```



F-Strings

- Όταν εργαζόμαστε με Λεξικά, θα πρέπει να βεβαιωθούμε ότι αν χρησιμοποιούμε Διπλά Εισαγωγικά (") με το F-String, τότε θα πρέπει να χρησιμοποιήσουμε Μονά Εισαγωγικά (') για τα Κλειδιά μέσα στο F-String και αντίστροφα.

Παράδειγμα:

```
info = { "Id": 100, "Name": "Admin" }
print( f"Id of {info['Name']} is {info['Id']}." )
```

F-Strings

- Όπως αναφέραμε και προηγουμένως, μπορούμε και να Μορφοποιήσουμε με συγκεκριμένο τρόπο τα δεδομένα κειμένου μας πριν τα εκτυπώσουμε στην οθόνη χρησιμοποιώντας τα F-Strings.
- Σε ένα F-String μπορούμε να ορίσουμε μέσα στις Θέσεις Περιεχομένου (Place Holders) παραμέτρους και να εμφανίσουμε τα δεδομένα μας με μια Συγκεκριμένη Μορφή.

F-Strings

- Έτσι μπορούν να δηλωθούν λεπτομερέστερες Θέσεις Περιεχομένου, όπως να περάσουμε παραμέτρους μορφοποίησης και εμφάνισης απλά προσθέτοντας με τα το όνομα της Μεταβλητής της Θέσεις Περιεχομένου, για παράδειγμα, τον χαρακτήρα της Άνω-Κάτω Τελείας (:).

Συντακτικό:

```
print( f"κείμενο {<μεταβλητή>:<παράμετρος>}")
```

F-Strings

Παράδειγμα, Εάν: salary = 250.34535

- Δεκαδικός (.) ακρίβειας τριών ψηφίων.

```
print( f"Your salary is: {salary:.3f}")
```



- Γέμισμα με κάτω παύλες (_) με το κείμενο κεντραρισμένο (^) σε πλάτος 11.

```
print( f"Your salary is: {salary:_^11}"
```



Ακέραιοι Αριθμοί

- Μπορούμε να τυπώνουμε Ακεραίους Αριθμούς σε διάφορα Αριθμητικά Συστήματα, με την χρήση των παρακάτω Τελεστών:

1. d : Για Δεκαδικούς Αριθμούς.
2. b : Για Δυαδικούς Αριθμούς.
3. o : Για Οκταδικούς Αριθμούς.
4. x : Για Δεκαεξαδικούς Αριθμούς.

Ακέραιοι Αριθμοί

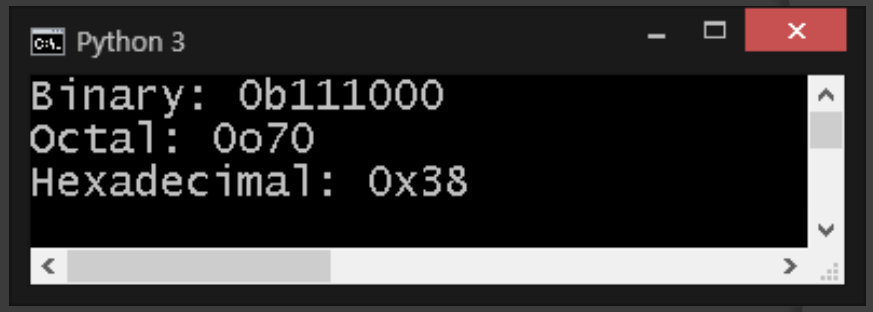
Παράδειγμα:

```
Number = 56
print( f"Decimal: {Number:d}" )
print( f"Binary: {Number:b}" )
print( f"Octal: {Number:o}" )
print( f"Hexadecimal: {Number:x}" )
```



Ακέραιοι Αριθμοί

- Προσθέτοντας τον χαρακτήρα της Δίεσης (#), εκτυπώνεται επιπλέον Πληροφορία που δείχνει σε ποιο Αριθμητικό Σύστημα ανήκει ο εκτυπωμένος αριθμός.



ΠΧ.:

```
Number = 56
print( f"Decimal: {Number:#d}" )
print( f"Binary: {Number:#b}" )
print( f"Octal: {Number:#o}" )
print( f"Hexadecimal: {Number:#x}" )
```

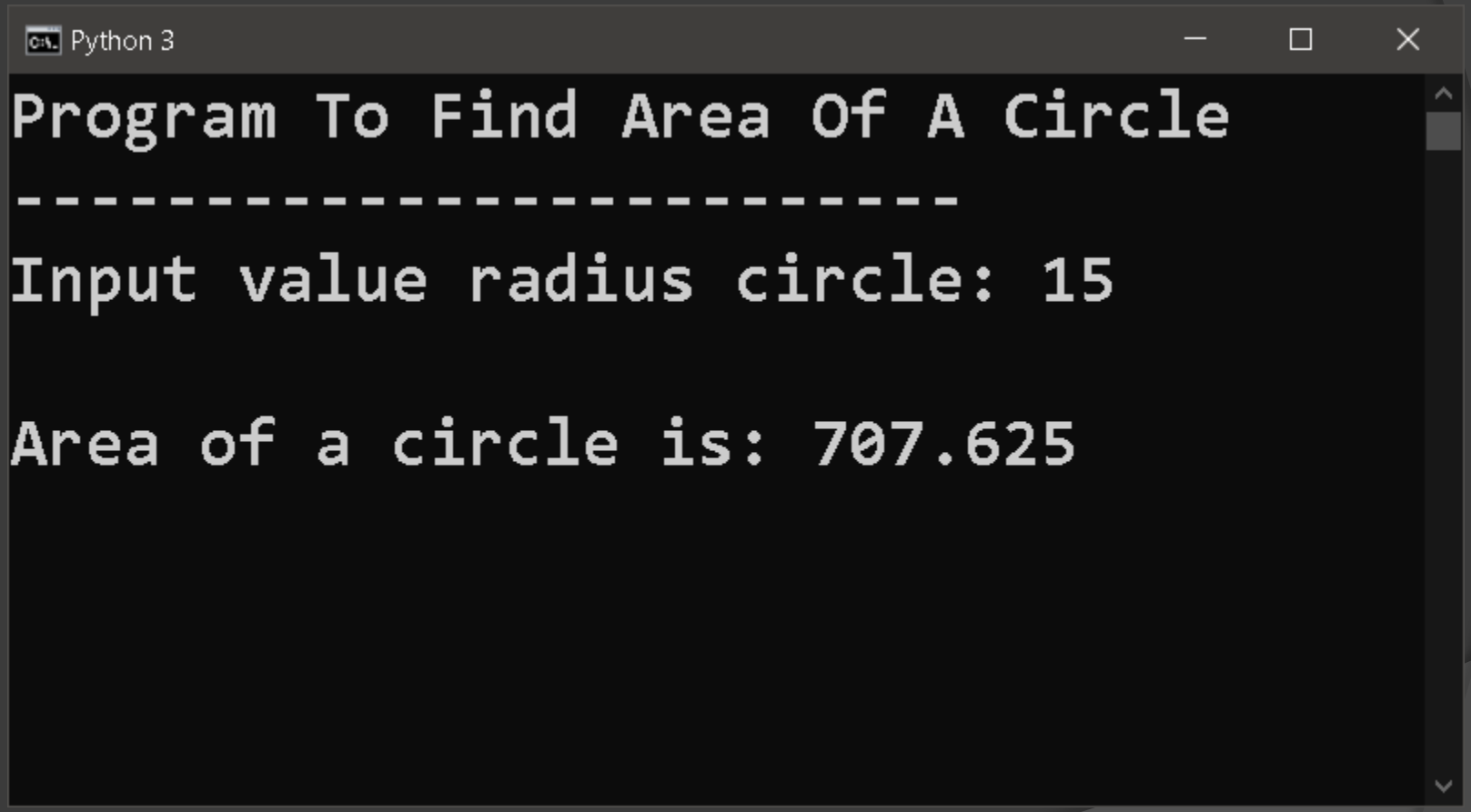
Άσκηση 1

- Δημιουργήστε ένα απλό πρόγραμμα που θα ζητά από τον χρήστη να εισάγει την ακτίνα ενός κύκλου και το πρόγραμμα θα υπολογίσει το εμβαδόν του κύκλου αυτού.

$$\text{Εμβαδόν} = (\pi * \text{ακτίνα}^2)$$

- Το αποτέλεσμα να εμφανίζετε με τρία μόνο δεκαδικά ψηφία.

Άσκηση 1


Python 3
Program To Find Area Of A Circle

Input value radius circle: 15
Area of a circle is: 707.625

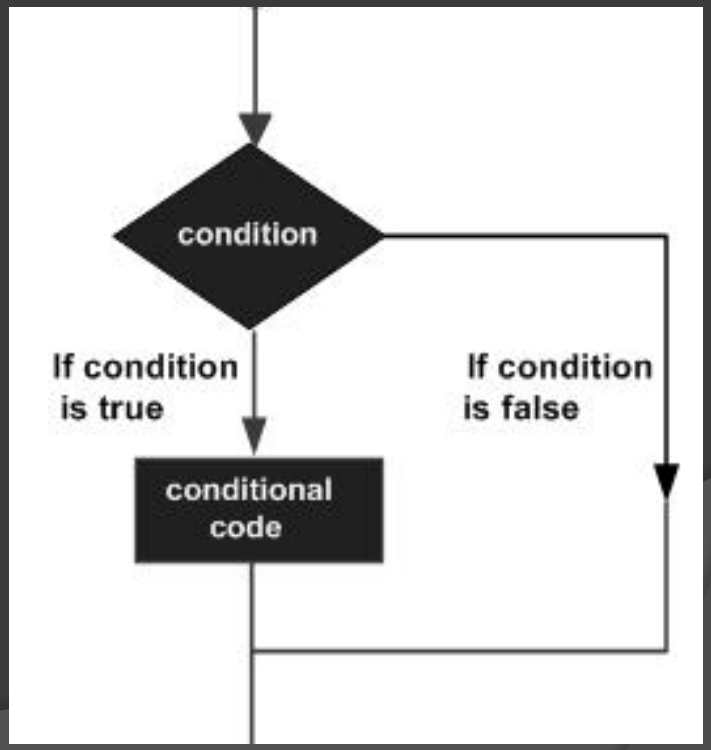
Η Δομή match

Λήψη Αποφάσεων

- Οι Δομές Λήψης Αποφάσεων απαιτούν από τον προγραμματιστή να καθορίσει μία ή περισσότερες προϋποθέσεις που θα Αξιολογηθούν ή θα Δοκιμαστούν από το πρόγραμμα, μαζί με μια δήλωση ή δηλώσεις που θα εκτελεστούν εάν η συνθήκη προσδιορίζεται ότι είναι αληθής και, προαιρετικά, άλλες δηλώσεις που θα εκτελεστούν εάν η συνθήκη προσδιορίζεται ότι είναι να είναι ψευδής.

Λήψη Αποφάσεων

- Ακολουθεί η γενική μορφή μιας τυπικής Δομής Λήψης Αποφάσεων που βρίσκεται στις περισσότερες Γλώσσες Προγραμματισμού:



Η Εντολή match

- Για πολλά χρόνια η Python δεν είχε μια δομή όπως η switch της C/C++ και της Java.
- Από την έκδωση 3.10 της Python και μετά, μπορούμε να χρησημοποιήσουμε την Δομή match.
- Η Δομή match λειτουργεί όπως η switch και χρησιμοποιείται όταν έχουμε πολλαπλές πίθανές τιμές για μια μεταβλητή και θέλουμε να εκτελεστούν διαφορετικές εντολές για κάθε τιμή.

Η Εντολή match

- Με την match δεν χρειάζεται να δημιουργούμε μεγάλες «αλυσίδες» εντολών if…elif...else και ο κώδικας μας παραμένει απλός και ευανάγνωστος.

Συντακτικό:

```
match value:
    case Τιμή1:  # Εντολές για την Τιμή 1
    case Τιμή2:  # Εντολές για την Τιμή 2
    …
   case _:  # Προεπιλεγμένες Εντολές
```

Η Εντολή match

- Για κάθε πιθανή τιμή της Μεταβλητής που ελέγχουμε με την match, δημιουργούμε μια διαφορετική απάντηση με τον δικό της κώδικα εκτέλεσης χρησιμοποιώντας μια Πρόταση case.
- Εάν η Μεταβλητή δεν είναι ίση με καμία από τις πιθανές τιμές που δίνουμε ανά περίπτωση, τότε ο κώδικας της Πρότασης case με τιμή _: θα εκτελεστεί αυτόματα από το πρόγραμμα.

Η Εντολή match

```
day = input("Enter a day:")
match day:
    case "Monday":
        print("It's the start of the workweek.")
    case  "Friday":
        print("Weekend is near!")
    case "Saturday" | "Sunday":  # | είναι το OR
        print("It's the weekend!")
    case _: print("It's a regular weekday.")
```

Η Εντολή match

```
day = int( input("Enter your Age:") )
match age:
    case 18:
        print("You are now an adult!")
    case 16:
        print("Sweet sixteen!")
    case _:
        print("Your age doesn't match any
                                    known patterns.")
```

Ο Τελεστής Pipe

- Η Δομή match προορίζονταν μόνο για την αντιστοίχιση των Μοτίβων και συγκεκριμένων Λέξεων-Κλειδιών ή Παραμέτρων σε ένα πρόγραμμα της Python.
- Παρόλα αυτά η Δομή match είναι αρκετά ευέλικτη έτσι ώστε να μπορείτε να χρησιμοποιήσετε τον Ιδικό Τελεστή pipe (|) για την υλοποίηση της λογικής πράξεις OR στην πρόταση case.
- Με αυτόν τον τρόπο μπορούμε να απλοποιήσουμε τον κώδικα μας αρκετά.

Ο Τελεστής Pipe

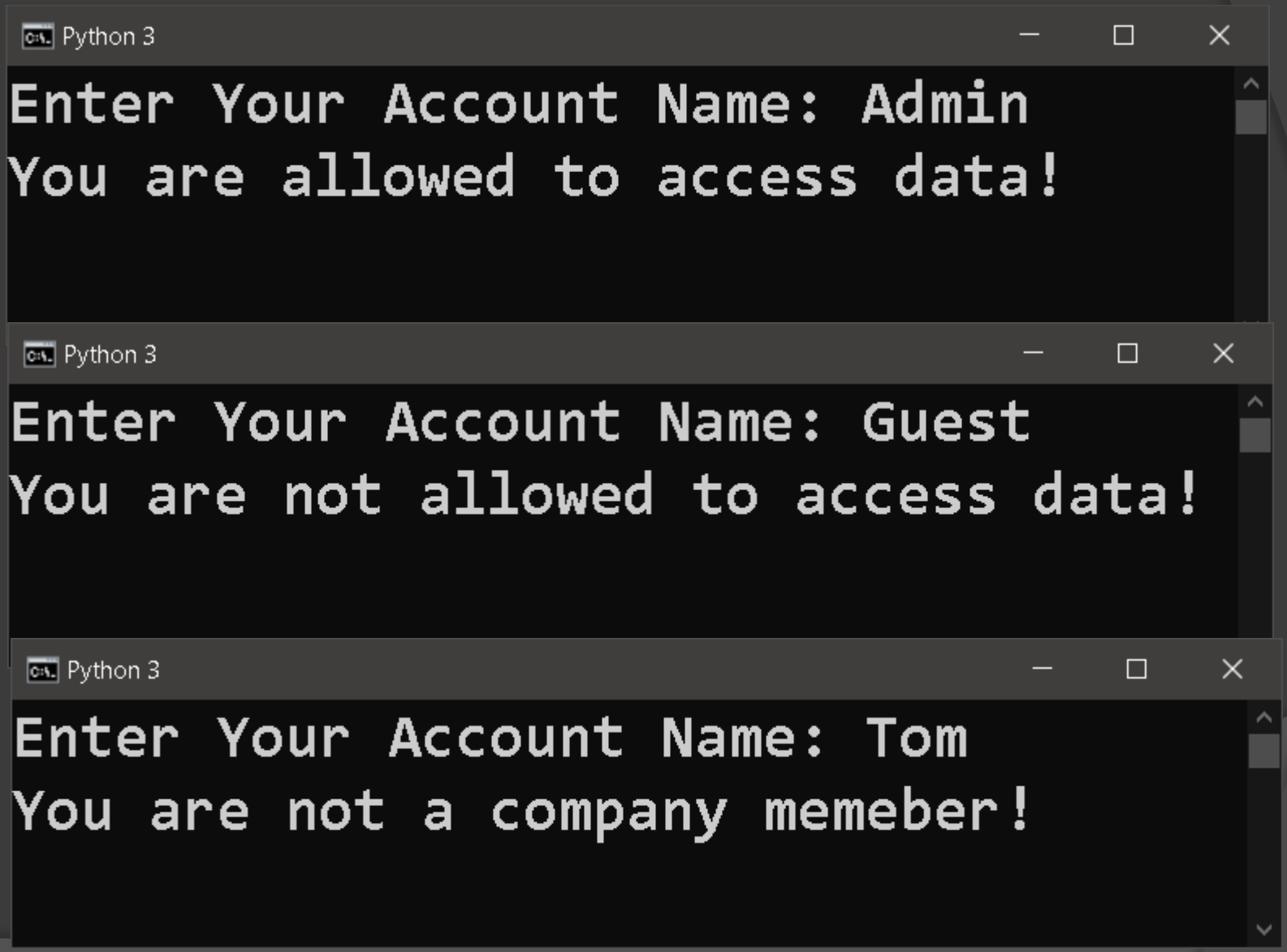
Συντακτικό:

```
case Τιμή1 | Τιμή2 | Τιμή3 | …:
```

Παράδειγμα:

```
match userName:
    case "User" |  "Guest" | "Temp" :
        print("You are not allowed to access data!")
    case "Root" | "Admin":
        print("You are allowed to access data!")
    case _ :
        print("You are not a company memeber!")
```

Ο Τελεστής Pipe



Η Συνθήκη if

- Μπορούμε επίσης να χρησιμοποιήσουμε τη συνθήκη if της Python, η οποία ουσιαστικά δεν είναι τίποτε άλλο από την απλή Δομή if για την αντιστοίχιση Μοτίβων μέσα στην Δομή match.
- Όταν μια περίπτωση Αντιστοιχιστεί, τότε μεταβαίνουμε στη συνθήκη if και αν η συνθήκη για την εντολή if αποδειχθεί αληθής, τότε επιλέγεται το μπλοκ κώδικα για εκτέλεση, διαφορετικά δεν εκτελείται.

Η Συνθήκη if

- Στο παρακάτω παράδειγμα, ελέγχουμε την περίπτωση αν το όνομα του λογαριασμού που έδωσε ο χρήστης αναφέρεται σε κάποια case της Δομής match.
- Εάν ναι, ελέγχει στη συνέχεια για την ύπαρξη του ονόματος στη Λίστα adminNames.
- Εάν ισχύει, τότε η συνθήκη για την πρόταση της if είναι η αληθής και θα εκτελεστεί το μπλοκ κώδικα, παρέχοντας του πρόσβαση στη Βάση Δεδομένων.

Η Συνθήκη if

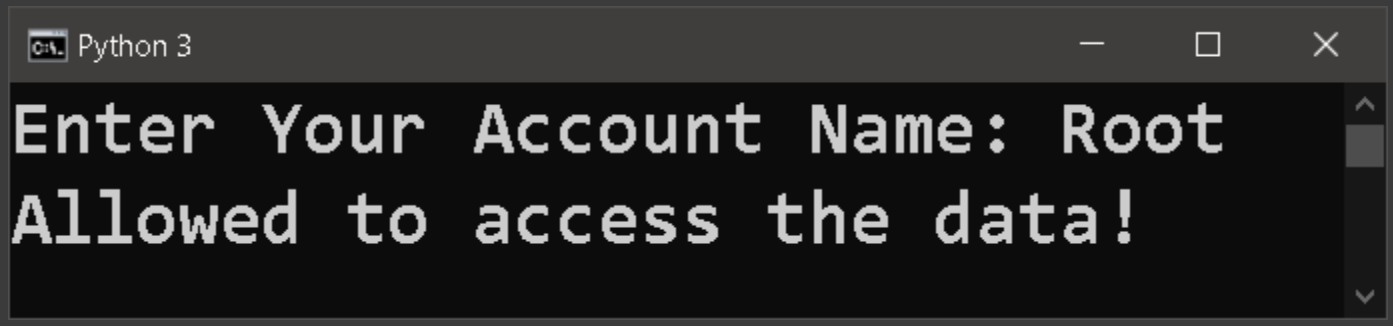
Συντακτικό:

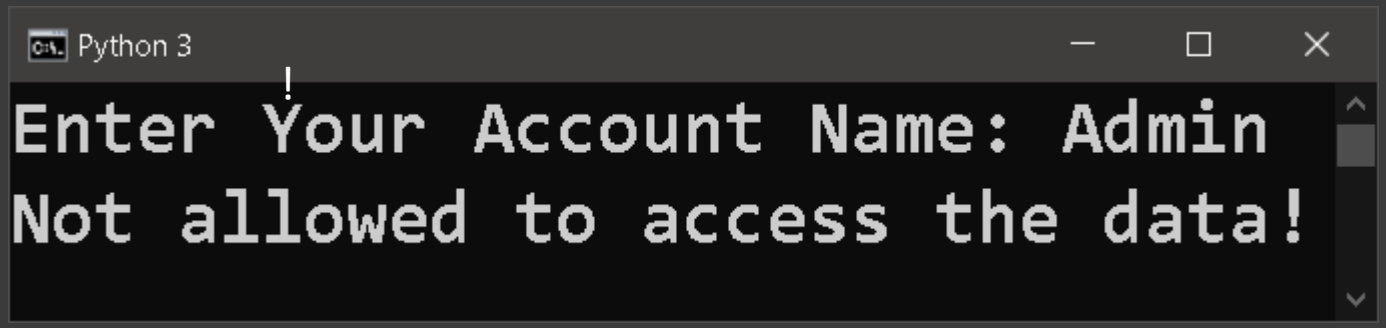
```
case Τιμή if <μεταβλητή> in <λίστα>:
```

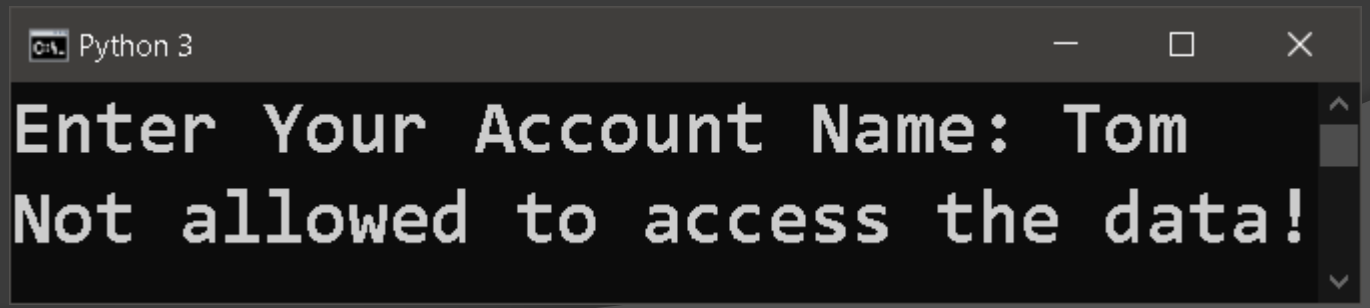
Παράδειγμα:

```
adminNames = ["Admin", "Root", "Supervisor"]
match userName:
    case "Root" if userName in adminNames:
        print("Allowed to access the data!")
    case _:
        print("Not allowed to access the data!")
```

Η Συνθήκη if


Python 3
Enter Your Account Name: Root
Allowed to access the data!


Python 3
Enter Your Account Name: Admin
Not allowed to access the data!


Python 3
Enter Your Account Name: Tom
Not allowed to access the data!

Αντιστοίχιση Τιμών

- Τέλος μπορούμε επίσης να αντιστοιχίσουμε τιμές χρησιμοποιώντας τη Δομή match σε συνδυασμό με τιμές από μια υπάρχουσα Δομή Ακολουθίας όπως το Λεξικό Δεδομένων.
- Σε αυτήν την περίπτωση στην συνθήκη case προσθέτουμε σαν Τιμή Ελέγχου το Κλειδί του Κόμβου με την αντίστοιχη τιμή του.

Συντακτικό:

```
case { "κλειδί": τιμή }
```

Αντιστοίχιση Τιμών

- Η Αντιστοίχιση είναι ένας κοινός τύπος δεδομένων στην Python και η Αντιστοίχιση μπορεί να χρησιμοποιηθεί με Λεξικά, δημιουργώντας Δομές που ελέγχουν για συγκεκριμένα Κλειδιά και Τιμές.

Παράδειγμα:

- Στο επόμενο παράδειγμα, η Συνάρτηση userID αντιστοιχεί με τη Δομή ενός Λεξικού με το όνομα user.

Αντιστοίχιση Τιμών

- Η πρώτη case αντιστοιχεί με ένα Λεξικό που περιέχει και τα δύο Κλειδιά "name" και "age", συνδέοντας τις αντίστοιχες τιμές τους με δύο Μεταβλητές.
- Σε αυτήν την περίπτωση τα ονόματα των Μεταβλητών μπορούν να διαφέρουν, αλλά συνηθίζεται να χρησιμοποιούμε τα ίδια τα ονόματα των Κλειδιών του κάθε Κόμβου.
- Έτσι αφού έχουμε τα Κλειδιά "name" και "age", θα χρησιμοποιήσουμε τα ονόματα Μεταβλητών name και age αντίστοιχα.

Αντιστοίχιση Τιμών

- Η δεύτερη case ταιριάζει με ένα Λεξικό που χρησιμοποιεί μόνο το Κλειδί "name".
- Ο χαρακτήρας μπαλαντέρ _ χρησιμοποιείται για να εντοπίσει οποιαδήποτε άλλη Δομή Λεξικού που δεν ταιριάζει με αυτά τα μοτίβα, που ορίσαμε πριν στις προηγούμενες case, εκτυπώνοντας στην οθόνη το μήνυμα "Unknown Entry".

Αντιστοίχιση Τιμών

```
def userID( user ):
    match user:
        case { "name": name, "age": age }:
            print( f"Name: {name}, Age: {age}" )

        case { "name": name }:
            print( f"Name: {name}" )
        case _:
            print( "Unknown Entry" )
...
```

Αντιστοίχιση Τιμών

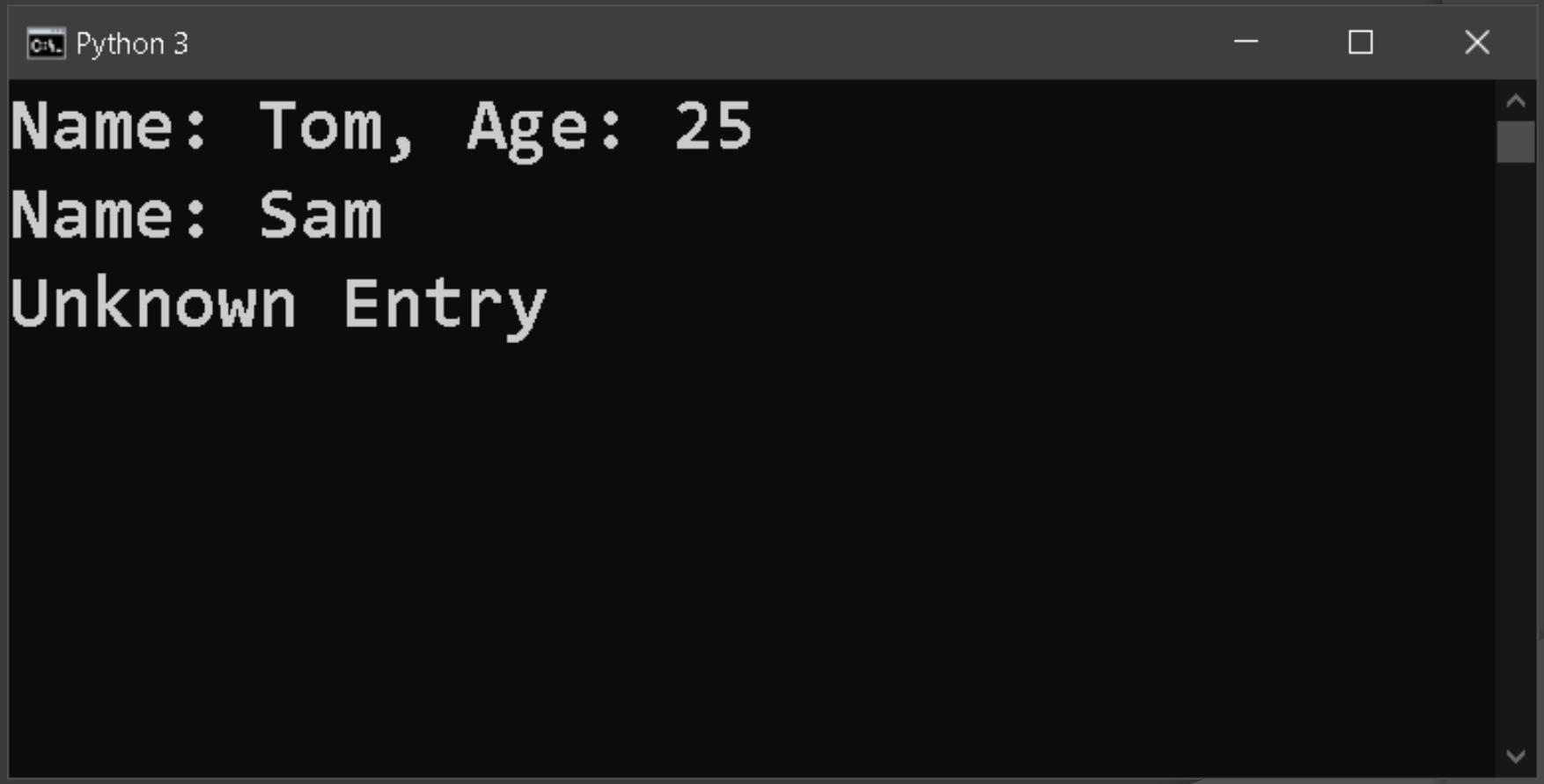
...

```
userID( {"name": "Tom", "age": 25} )

userID( {"name": "Sam"} )

userID( {"level": "Admin"} )
```

Αντιστοίχιση Τιμών



Επιλογή Τύπου Δεδομένων

- Επειδή η Python δεν επιβάλει στις Μεταβλητές να δηλώνουν τον Τύπου Δεδομένων τους κατά την δημιουργία τους, η match μας δίνει την δυνατότητα να εκτελούμε διαφορετικό κώδικα ανά Τύπου Δεδομένων.
- Αυτό είναι ιδιαίτερα χρήσιμο ιδικά στις Συναρτήσεις όπου ο χρήστης μπορεί να περνά σαν παραμέτρους τιμές οποιουδήποτε Τύπου Δεδομένων.

Επιλογή Τύπου Δεδομένων

- Στο προηγούμενο εξάμηνο μάθαμε ότι για κάθε Τύπο Δεδομένων, η Python μας δίνει και μια Συνάρτηση η οποία μας παρέχει την δυνατότητα να διαχειριστούμε τις τιμές του συγκεκριμένου Τύπου.
- Οι Συναρτήσεις αυτές είναι:

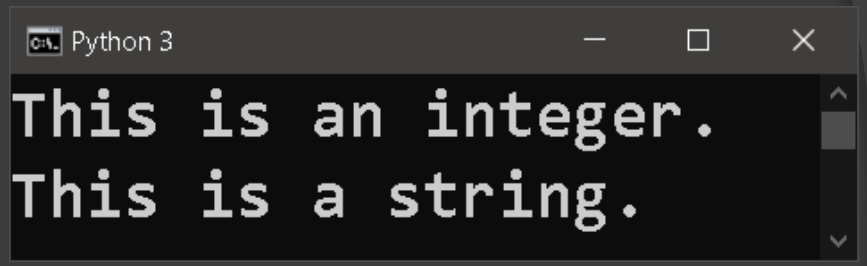
Κείμενο:	str()
Αριθμοί:	int(), float(), complex()
Ακολουθίες:	list(), tuple(), range()
Αντιστοίχιση:	dict()
Σετ:	set(), frozenset()
Boolean:	bool()
Δυαδικά:	bytes(), bytearray(), memoryview()

Επιλογή Τύπου Δεδομένων

- Στο επόμενο παράδειγμα θα χρησιμοποιήσουμε αυτές τις Συναρτήσεις για να εκτελέσουμε διαφορετικό κώδικα ανά περίπτωση:

Παράδειγμα:

```
def dataTypes(value):
    match value:
        case int():    return "This is an integer."
        case str():    return "This is a string."
        case list():    return "This is a list."
dataTypes(23)
dataTypes("Admin")
```



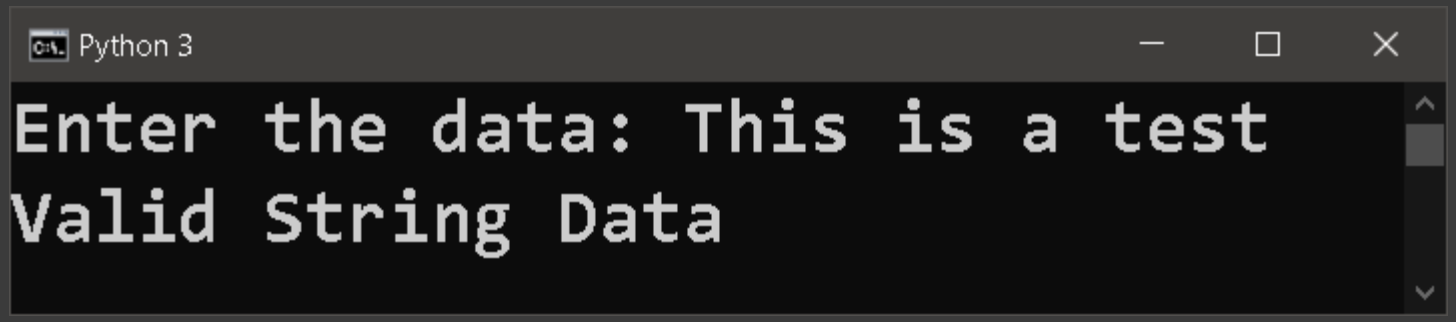
Άσκηση 2

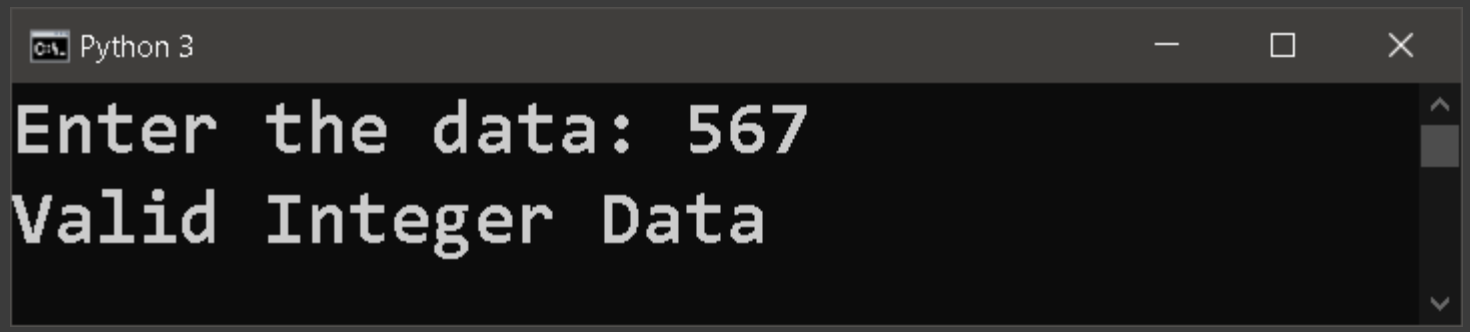
- Δημιουργήστε μια εφαρμογή Python όπου ο χρήστης θα περνά από το πληκτρολόγιο δεδομένα.
- Το πρόγραμμα θα περνά τα δεδομένα σε μια Συνάρτηση με το όνομα "validation()" και η Συνάρτηση θα ελέγχει μέσω μιας Δομής match την ορθότητα των δεδομένων.
- Οι κανόνες είναι οι παρακάτω:

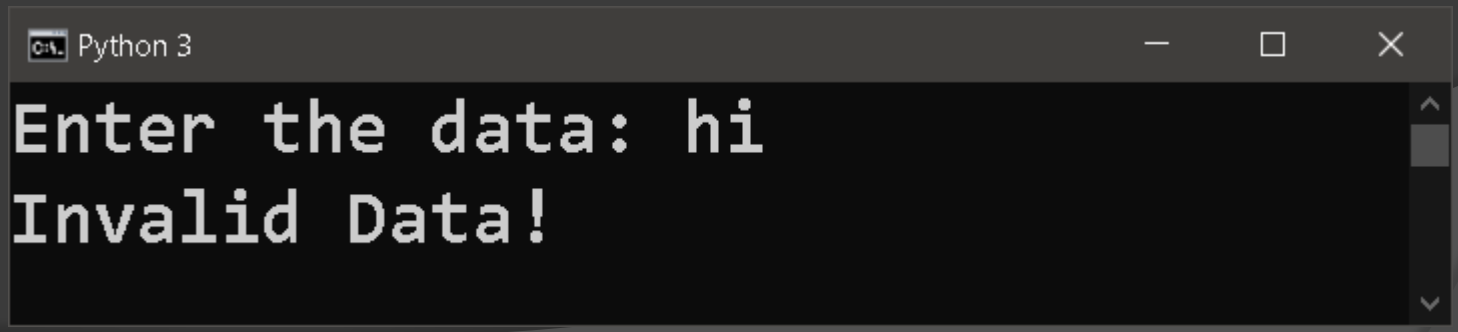
Άσκηση 2

1. Εάν τα δεδομένα είναι σε μορφή κειμένου και ο αριθμός χαρακτήρων που περιέχει είναι μεγαλύτερος ή ίσος με το 10, τότε τα δεδομένα θα γίνονται αποδεκτά.
2. Εάν τα δεδομένα είναι σε μορφή αριθμού και ο αριθμός αυτός είναι μεγαλύτερος ή ίσος με το 50, τότε τα δεδομένα αυτά θα γίνονται αποδεκτά.
3. Σε όλες τις άλλες περιπτώσεις ένα μήνυμα λάθους θα εμφανίζεται και τα δεδομένα αυτά δεν θα γίνονται αποδεκτά.

Άσκηση 2


Python 3
Enter the data: This is a test
Valid String Data


Python 3
Enter the data: 567
Valid Integer Data


Python 3
Enter the data: hi
Invalid Data!

Συναρτήσεις Γεννήτριας

Συναρτήσεις Γεννήτριας

- Η Python μας δίνει την δυνατότητα να χρησιμοποιήσουμε μια Γεννήτρια για να δημιουργήσουμε τη δική μας Συνάρτηση Βρόχου.
- Μια Γεννήτρια (Generator) είναι ένας ειδικός τύπος συνάρτησης που δεν επιστρέφει μια μεμονωμένη τιμή, αντίθετα, επιστρέφει ένα αντικείμενο που περιέχει μια ακολουθία από πολλές και διαφορετικές τιμές.

Συναρτήσεις Γεννήτριας

- Σε αυτό το αντικείμενο, που ονομάζεται Lazy Iterator, μπορούμε να εφαρμόσουμε επαναληπτικές ενέργειες σαν να ήταν μια απλή Λίστα Τιμών.
- Ωστόσο, σε αντίθεση με τις Λίστες, οι Lazy Iterators δεν αποθηκεύουν τα περιεχόμενά τους στη μνήμη του υπολογιστή.

Συναρτήσεις Γεννήτριας

- Οι Συναρτήσεις Γεννήτριας χρησιμεύουν παράδειγμα όταν:
  1. Θέλουμε να χειριστούμε ένα σύνολο δεδομένων τόσο μεγάλο που να κατακλύζει τη μνήμη του μηχανήματός μας.
  2. Όταν έχουμε μια πολύπλοκη συνάρτηση που πρέπει να διατηρεί την εσωτερική της κατάσταση κάθε φορά που καλείται, αλλά να είναι πολύ μικρή για να δικαιολογήσει τη δημιουργία της δικής της Κλάσης.

Συναρτήσεις Γεννήτριας

- Οι Συναρτήσεις Γεννήτριας χρησιμεύουν παράδειγμα όταν:
  3. Σε προγράμματα που διαχειρίζονται αρχεία και περιεχόμενα φακέλων.
  4. Σε προγράμματα που χρησιμοποιούν αναδρομή και αναδρομικές συναρτήσεις.
  5. Και σε πολλές ακόμα περιπτώσεις.

Συναρτήσεις Γεννήτριας

- Σε μια Συνάρτηση Γεννήτριας, χρησιμοποιούμε την εντολή yield αντί για την εντολή return.

Συντακτικό:

```
def όνομα():
    yield τιμή-1
    yield τιμή-2
    yield τιμή-3
    …
```

Συναρτήσεις Γεννήτριας

- Η διαφορά μεταξύ της yield και της return είναι ότι η yield επιστρέφει μια τιμή και σταματά προσωρινά την εκτέλεση της συνάρτησης διατηρώντας την εσωτερική της κατάσταση ενώ η return επιστρέφει μια τιμή και τερματίζει την εκτέλεση της συνάρτησης.

Συναρτήσεις Γεννήτριας

- Στο παρακάτω παράδειγμα έχουμε τη υλοποίηση μας απλής Συνάρτησης Γεννήτριας.

```
def mygenerator():
    print('First item')
    yield 10
    print('Second item')
    yield 20
    print('Last item')
    yield 30
```

Συναρτήσεις Γεννήτριας

- Στο παραπάνω παράδειγμα, η συνάρτηση mygenerator() είναι μια Συνάρτηση Γεννήτριας που χρησιμοποιεί την εντολή yield αντί για την εντολή return.
- Έτσι, το μόνο που έχουμε να κάνουμε τώρα είναι να χρησιμοποιήσουμε μια μεταβλητή για να αποθηκεύσουμε στην μνήμη το αντικείμενο που επιστρέφει η mygenerator() με τις αντίστοιχες τιμές.

Συναρτήσεις Γεννήτριας

Παράδειγμα:

```
value = mygenerator()
print(value)
```

- Το αποτέλεσμα θα είναι το παρακάτω:



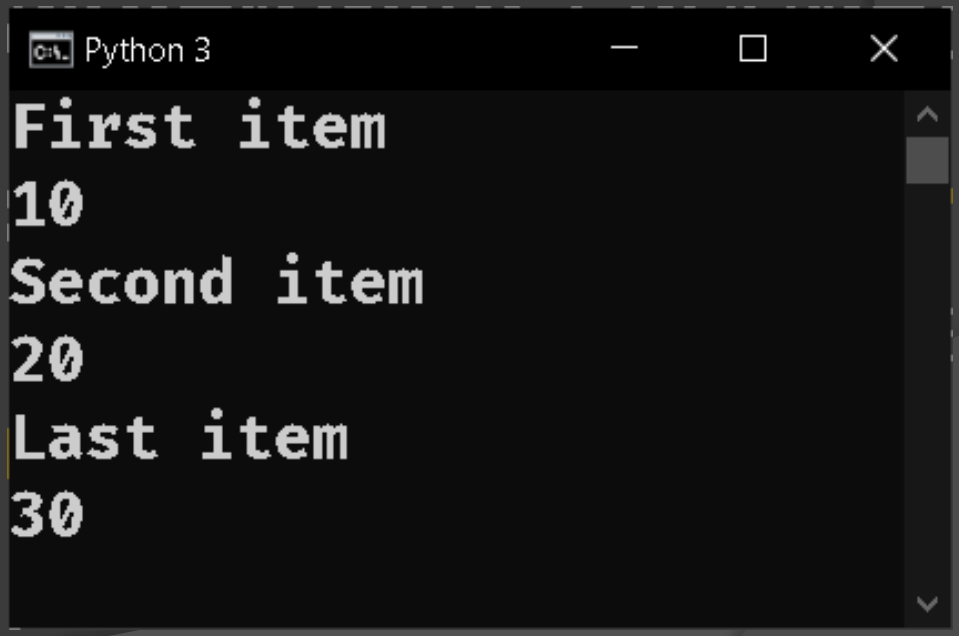
- Μας ενημερώνει για τον τύπο και την θέση μνήμης που καταλαμβάνει στην RAM.

Συναρτήσεις Γεννήτριας

- Για να δούμε στην οθόνη τις αντίστοιχες τιμές που περιέχει το αντικείμενο θα χρειαστεί να χρησιμοποιήσουμε έναν βρόχο.

Παράδειγμα:

```
for i in mygenerator():
    print(i)
```

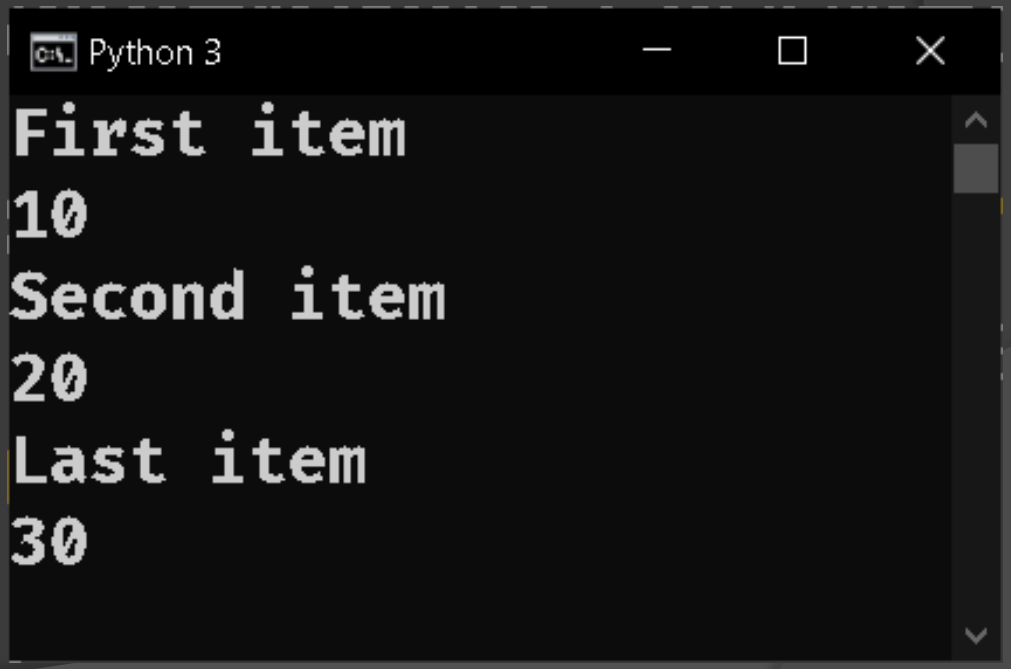


Συναρτήσεις Γεννήτριας

- Εναλλακτικά μπορούμε να χρησιμοποιήσουμε και την μέθοδο της Python με το όνομα next() για να πάρουμε μία-μία τις τιμές.

Παράδειγμα:

```
print ( next(values) )
print ( next(values) )
print ( next(values) )
```

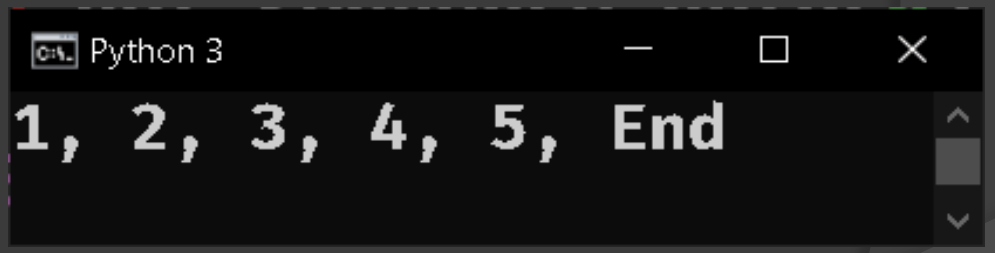


Συναρτήσεις Γεννήτριας

- Η Συναρτήσεις Γεννήτριας μπορούν επίσης να χρησιμοποιήσουν τον βρόχο for.

Παράδειγμα:

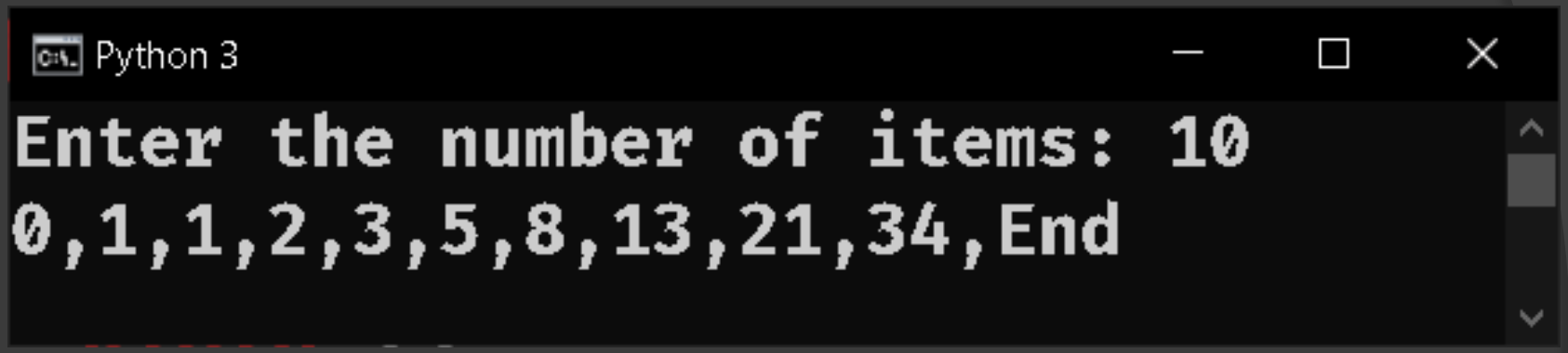
```
def get_sequence_upto(x):
    for i in range( 1, x ):
        yield i
for j in get_sequence_upto(6):
        print( j, end =", " )
print("End")
```

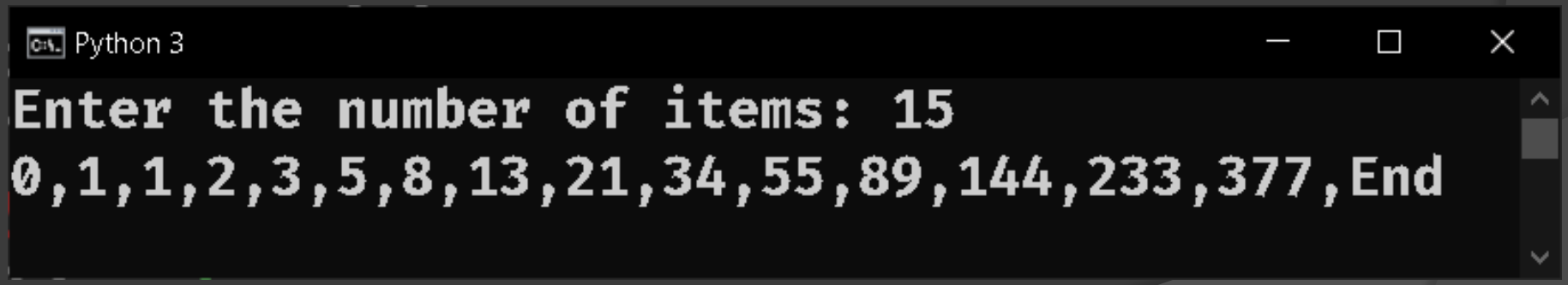


Άσκηση 3

- Γράψτε ένα πρόγραμμα Python πού θα εμφανίζει τους X πρώτους αριθμούς στην Ακολουθία Fibonacci χρησιμοποιώντας μια Συνάρτηση Γεννήτριας.
- Το πρόγραμμα θα ζητά από τον χρήστη να εισάγει τον μέγιστο αριθμό στοιχείων, δηλαδή την Τιμή του X.
- Στην Ακολουθία Fibonacci ο κάθε αριθμός είναι ίσος με το άθροισμα των δύο προηγούμενων αριθμών ξεκινώντας από τους αριθμούς 0 και 1.

Άσκηση 3


Python 3
Enter the number of items: 10
0,1,1,2,3,5,8,13,21,34,End


Python 3
Enter the number of items: 15
0,1,1,2,3,5,8,13,21,34,55,89,144,233,377,End